JALA Academy
Assignment 7

Inheritance & Polymorphism

Python solution reference • Code examples are organized by the assignment questions.

How to use this document

Save the code examples into a .py file and run with Python 3. Where a program asks for input, enter values when prompted. For file-handling exercises, create the named text file in the same folder first.

1–7. Classes A, B and C

```
class A:
    def __init__(self):
        print("A constructor")
        self.level = "A"

    def a_one(self): print("A specific method 1")
    def a_two(self): print("A specific method 2")
    def show(self): print("Method from A")

class B(A):
    def __init__(self):
        super().__init__()
        print("B constructor")
        self.level = "B"

    def b_one(self): print("B specific method 1")
    def b_two(self): print("B specific method 2")
    def show(self):
        print("Method from B")
        super().show()

class C(B):
    def __init__(self):
        super().__init__()
        print("C constructor")
        self.level = "C"

    def c_one(self): print("C specific method 1")
    def c_two(self): print("C specific method 2")
    def show(self):
        print("Method from C")
        super().show()

a = A()
b = B()
c = C()
a.a_one(); a.a_two(); a.show()
b.a_one(); b.b_one(); b.b_two(); b.show()
c.a_one(); c.b_one(); c.c_one(); c.c_two(); c.show()

# Runtime polymorphism through a superclass-typed reference
for ref in (b, c):
    ref.show()

print("Levels:", a.level, b.level, c.level)
# Creating C calls constructors in order A, B, C.

class Vehicle:
    def start_engine(self): print("Vehicle engine starts")
class Car(Vehicle):
    def start_engine(self):
        super().start_engine()
        print("Car is ready to drive")
class ElectricCar(Car):
    def start_engine(self):
        super().start_engine()
        print("Electric motor is ready")
```

```
ElectricCar().start_engine()
```